

WEB DEVELOPMENT

Complete Study Notes

Unit I: Internet Basics & Protocols

Unit II: HTML — HyperText Markup Language

Unit III: CSS — Cascading Style Sheets

Unit IV: JavaScript & DOM

Unit V: PHP — Back-End Technologies

Prepared for Oral Examination | Beginner to Intermediate Level

Covers Every Keyword, Tag, Property & Concept

TABLE OF CONTENTS

Unit	Topic	Page
I	Internet Basics: Protocols, Servers, Clients, Web Dev	3
II	HTML: Tags, Attributes, Forms, Tables, Images, Links	7
III	CSS: Types, Selectors, Box Model, Flexbox, Properties	18
IV	JavaScript: Variables, Functions, DOM, Events	25
V	PHP: Syntax, Loops, Functions, Arrays, Strings	33

UNIT I — INTRODUCTION TO INTERNET AND WEB DEVELOPMENT

1.1 What is the Internet?

The Internet is a global network of computers and devices connected together using standardised communication protocols. It allows millions of computers to exchange data, share resources, and communicate in real time.

EXAM: The Internet is NOT the same as the World Wide Web. The Internet is the infrastructure (wires, routers, servers). The Web is a service that runs ON the Internet.

1.2 What is the World Wide Web (WWW)?

The World Wide Web (WWW or 'the Web') is a system of interlinked hypertext documents (web pages) accessed via the Internet using a web browser. It was invented by Tim Berners-Lee in 1989.

- Web pages are written in HTML.
- Web pages are linked together using hyperlinks.
- Web pages are accessed using a URL (Uniform Resource Locator).

1.3 Protocols

A **protocol** is a set of rules that govern how data is transmitted over a network. Think of it as the 'language' that computers speak to each other.

Key Protocols:

Protocol	Full Name	Purpose
HTTP	HyperText Transfer Protocol	Transfer web pages (unsecured)
HTTPS	HTTP Secure	Secure (encrypted) web transfer — uses SSL/TLS
FTP	File Transfer Protocol	Transfer files between computers
SMTP	Simple Mail Transfer Protocol	Send emails
POP3	Post Office Protocol v3	Receive/download emails
IMAP	Internet Message Access Protocol	Access emails on mail server
TCP	Transmission Control Protocol	Reliable, ordered data delivery
IP	Internet Protocol	Addressing and routing packets
DNS	Domain Name System	Translates domain names to IP addresses

EXAM: HTTP uses port 80. HTTPS uses port 443. FTP uses port 21. SMTP uses port 25.

NOTE: HTTP is stateless — every request is independent. The server does not remember previous requests.

1.4 What is HTTP? (Deep Dive)

HTTP (HyperText Transfer Protocol) is the foundation of data communication on the Web.

HTTP Request Methods:

- **GET** — Requests data from a server (e.g., loading a webpage). Data visible in URL.
- **POST** — Sends data to a server (e.g., submitting a form). Data hidden in body.
- **PUT** — Updates existing data on server.
- **DELETE** — Deletes data from server.

HTTP Status Codes:

- **200 OK** — Request successful.
- **301 Moved Permanently** — Resource moved.
- **404 Not Found** — Resource does not exist.
- **500 Internal Server Error** — Server-side problem.

EXAM: 200 = OK, 404 = Not Found, 500 = Server Error — must know these!

1.5 Servers and Clients

The Internet follows a **Client-Server Architecture**.

Client:

A client is any device (computer, phone, tablet) or software (browser) that REQUESTS services or resources from a server.

- Examples: Chrome, Firefox, Edge (web browsers).
- The client sends an HTTP request and waits for a response.
- The client renders (displays) the HTML/CSS/JS received.

Server:

A server is a computer or software that PROVIDES services, files, or data to clients.

- Examples: Apache, Nginx, Microsoft IIS.
- The server listens for requests, processes them, and sends back a response.
- Servers run 24/7 and can handle thousands of simultaneous connections.

NOTE: Case Study: Students (users) act as clients. They open a browser, type a URL, and their browser sends an HTTP GET request to the web server. The server sends back the HTML page, and the browser displays it.

Functions of a Client:

- Send HTTP requests to the server.
- Receive and display the response (render HTML/CSS).
- Execute client-side scripts (JavaScript).
- Store cookies locally.

Functions of a Server:

- Listen for incoming client requests.
- Process the request (execute server-side scripts like PHP).
- Query databases if needed.
- Send back an HTTP response with the requested data.

1.6 URL Structure

```
https://www.example.com:8080/path/page.html?name=John#section1
```

Breaking it down:

- **https://** — Protocol (scheme)
- **www.example.com** — Domain name (host)
- **:8080** — Port number (optional; default 80 for HTTP, 443 for HTTPS)
- **/path/page.html** — Path to the resource
- **?name=John** — Query string (GET parameters)
- **#section1** — Fragment (anchor to section on page)

1.7 Web Development Overview

Web Development is the process of building and maintaining websites/web applications.

Front-End (Client Side):

- What the user SEES and INTERACTS with.
- Technologies: HTML, CSS, JavaScript.

Back-End (Server Side):

- The behind-the-scenes logic, database, and server.
- Technologies: PHP, Python, Node.js, MySQL.

Full-Stack:

- A developer who works on both front-end and back-end.

Static vs Dynamic Websites:

- **Static Website:** Pre-built HTML files. Content is same for everyone. No database needed. Fast and simple.
- **Dynamic Website:** Content is generated on-the-fly based on user input, database, etc. Uses server-side scripting (PHP, etc.). Example: Facebook, Amazon.

UNIT II — HYPER TEXT MARKUP LANGUAGE (HTML)

2.1 What is HTML?

HTML stands for **HyperText Markup Language**. It is the standard language used to create web pages. HTML describes the STRUCTURE and CONTENT of a webpage using elements called **tags**.

- **HyperText** = Text that contains links to other texts.
- **Markup** = Annotating text with tags to define structure.
- **Language** = A set of rules for writing.

NOTE: HTML is NOT a programming language. It is a markup language. It has no logic, loops, or conditions.

EXAM: HTML was created by Tim Berners-Lee. Current version is HTML5 (released 2014).

2.2 Basic Structure of an HTML Document

Every HTML page follows this exact structure:

```
<!DOCTYPE html> <html lang="en"> <head> <meta charset="UTF-8"> <meta name="viewport"
content="width=device-width, initial-scale=1.0"> <title>My First Web Page</title>
</head> <body> <h1>Hello World!</h1> <p>This is my first paragraph.</p> </body>
</html>
```

Line-by-line explanation:

- **<!DOCTYPE html>** — Declaration. Tells the browser this is an HTML5 document. NOT a tag — it's an instruction. MUST be the very first line. Case-insensitive.
- **<html>** — The root element. All other elements go inside this. The **lang** attribute sets the language.
- **<head>** — Contains meta-information about the document (not visible on screen).
- **<meta charset='UTF-8'>** — Defines character encoding. UTF-8 supports almost all characters and symbols.
- **<meta name='viewport'>** — Makes the page mobile-responsive.
- **<title>** — Sets the title shown on the browser tab.
- **<body>** — Contains all VISIBLE content of the web page.

EXAM: EXAM: <!DOCTYPE html> is NOT an HTML tag. It is a document type declaration. It tells the browser which HTML version to use.

2.3 HTML Tags, Elements and Attributes

Tag: A keyword enclosed in angle brackets. Example: **<p>**

Element: An opening tag + content + closing tag. Example: **<p>Hello</p>**

Attribute: Extra information inside the opening tag. Example: ****

```
<tagname attribute="value">Content goes here</tagname>
```

Self-Closing Tags (Void Elements):

These tags do not have closing tags because they have no content:

```
<br> (line break) <hr> (horizontal rule / line) <img> (image)
<input> (form input) <meta> (metadata) <link> (external resource
link)
```

**EXAM: EXAM:
 inserts a line break. <hr> inserts a horizontal line. Both are self-closing (void) tags.**

2.4 Text Formatting Tags

Heading Tags:

```
<h1>Largest Heading</h1> <h2>Second Heading</h2> <h3>Third Heading</h3> <h4>Fourth
Heading</h4> <h5>Fifth Heading</h5> <h6>Smallest Heading</h6>
```

NOTE: h1 is most important (largest), h6 is least important (smallest). Search engines give h1 the highest priority for SEO.

Paragraph and Text Tags:

```
<p>This is a paragraph.</p> <b>Bold text</b> (just visually bold) <strong>Important
bold</strong> (semantically important) <i>Italic text</i> (just visually italic)
<em>Emphasized italic</em> (semantically emphasized) <u>Underlined text</u>
<s>Strikethrough</s> <mark>Highlighted text</mark> <small>Smaller text</small>
<big>Bigger text</big> <sub>Subscript</sub> e.g. H<sub>2</sub></sub>0
<sup>Superscript</sup> e.g. x<sup>2</sup> <br> (Line break – goes to next line) <hr>
(Horizontal rule – draws a line) <pre>Preformatted text – preserves spaces and
newlines</pre> <code>Inline code snippet</code> <blockquote>Block quotation –
indented quote</blockquote>
```

Container/Semantic Tags:

```
<div> Block-level container (division). No visual effect. <span> Inline container. No
visual effect. <header> Page or section header <footer> Page or section footer <nav>
Navigation menu <main> Main content area <article> Independent content (blog post,
news) <section> Thematic section of content <aside> Sidebar / related content
```

EXAM: div = block-level (starts new line). span = inline (stays on same line). This is a very common exam question!

2.5 Images

```

```

Attributes of :

- **src** (source) — REQUIRED. Path to the image file. Can be relative or absolute URL.
- **alt** (alternate text) — REQUIRED. Description of image if it cannot be displayed. Important for accessibility and SEO.
- **width** — Width of image in pixels or percentage.
- **height** — Height of image in pixels.
- **title** — Tooltip text shown on hover.

Image file formats:

- **JPEG / JPG** — Best for photographs. Compressed.
- **PNG** — Best for graphics with transparency.
- **GIF** — Supports animation.
- **SVG** — Scalable vector graphics. Never loses quality.
- **WebP** — Modern format, smaller size, high quality.

Relative vs Absolute Paths:

```
<!-- Relative path – relative to current file location -->   <!--  
Absolute path – full URL --> 
```

2.6 Hyperlinks

```
<a href="https://www.google.com">Click here to visit Google</a>
```

The anchor tag `<a>` creates a hyperlink. The **href** attribute specifies the destination.

Types of Links:

```
<!-- External link (different website) --> <a href="https://www.google.com"  
target="_blank">Google</a> <!-- Internal link (same website) --> <a  
href="about.html">About Us</a> <!-- Email link --> <a  
href="mailto:info@example.com">Send Email</a> <!-- Phone link --> <a  
href="tel:+911234567890">Call Us</a> <!-- Anchor link (jumps to section on same page)  
--> <a href="#section2">Go to Section 2</a> <h2 id="section2">This is Section 2</h2>  
<!-- Link with no destination --> <a href="#">Placeholder link</a>
```

Important Attributes of `<a>`:

- **href** — Destination URL.
- **target='_blank'** — Opens link in NEW tab/window.
- **target='_self'** — Opens link in SAME tab (default).
- **title** — Tooltip text on hover.
- **rel='noopener'** — Security: prevents new tab from accessing original page.

EXAM: target='_blank' opens in new tab. target='_self' opens in same tab.

2.7 Image Maps

An image map allows you to create clickable areas (hotspots) on an image. Different areas of the same image can link to different URLs.

```
 <map name="worldmap">  
<!-- Rectangle area --> <area shape="rect" coords="0,0,100,100" href="europe.html"  
alt="Europe"> <!-- Circle area --> <area shape="circle" coords="200,150,50"  
href="africa.html" alt="Africa"> <!-- Polygon (irregular shape) --> <area  
shape="poly" coords="10,20,30,40,50,20" href="asia.html" alt="Asia"> <!-- Default  
area (entire image) --> <area shape="default" href="world.html" alt="World"> </map>
```


Shapes in image maps:

- **rect** — Rectangle. coords = x1,y1,x2,y2 (top-left and bottom-right corners).
- **circle** — Circle. coords = x,y,radius (center x, center y, radius).
- **poly** — Polygon. coords = x1,y1,x2,y2,x3,y3,... (multiple points).
- **default** — Entire image.

EXAM: usemap='#name' in img connects to <map name='name'>. The # is required!

2.8 Tables

```
<table border="1" cellpadding="5" cellspacing="0"> <caption>Student Marks</caption>
<thead> <tr> <th>Name</th> <th>Subject</th> <th>Marks</th> </tr> </thead> <tbody>
<tr> <td>Alice</td> <td>Math</td> <td>95</td> </tr> <tr> <td>Bob</td>
<td>Science</td> <td>88</td> </tr> </tbody> <tfoot> <tr> <td colspan="2">Average</td>
<td>91.5</td> </tr> </tfoot> </table>
```

Table Tags Explained:

- **<table>** — Container for the entire table.
- **<caption>** — Title/caption displayed above the table.
- **<thead>** — Groups the header rows.
- **<tbody>** — Groups the body rows.
- **<tfoot>** — Groups the footer rows.
- **<tr>** — Table Row. Contains cells.
- **<th>** — Table Header cell. Bold and centered by default.
- **<td>** — Table Data cell. Regular content.

Important Table Attributes:

- **border** — Width of table border in pixels.
- **cellpadding** — Space INSIDE each cell (between content and border).
- **cellspacing** — Space BETWEEN cells.
- **colspan** — Merges columns. colspan='2' spans 2 columns.
- **rowspan** — Merges rows. rowspan='2' spans 2 rows.
- **width / height** — Table or cell dimensions.
- **align** — Text alignment: left, right, center.
- **bgcolor** — Background color.

Nested Tables:

A table inside a table cell. Used for complex layouts.

```
<table border="1"> <tr> <td>Outer Cell 1</td> <td> <!-- Nested Table inside td -->
<table border="1"> <tr> <td>Inner A</td> <td>Inner B</td> </tr> </table> </td> </tr>
</table>
```

EXAM: colspan merges COLUMNS horizontally. rowspan merges ROWS vertically.

2.9 Lists

```
<!-- Unordered List (bullet points) --> <ul> <li>Apple</li> <li>Banana</li>
<li>Mango</li> </ul> <!-- Ordered List (numbered) --> <ol> <li>First step</li>
<li>Second step</li> <li>Third step</li> </ol> <!-- Definition List --> <dl>
<dt>HTML</dt> <dd>HyperText Markup Language</dd> <dt>CSS</dt> <dd>Cascading Style
Sheets</dd> </dl> <!-- Nested List --> <ul> <li>Fruits <ul> <li>Apple</li>
<li>Mango</li> </ul> </li> <li>Vegetables</li> </ul>
```

- **ul** — Unordered list. Default bullets.
- **ol** — Ordered list. Numbered 1,2,3.
- **li** — List item. Goes inside ul or ol.
- **dl** — Definition list. **dt** = term, **dd** = description.

NOTE: ol attribute type: type='1' (numbers), type='A' (uppercase), type='a' (lowercase), type='I' (Roman numerals).

2.10 HTML Forms — Case Study

Forms collect data from users. The Case Study: Create a Feedback Form using HTML form elements.

```
<form action="submit.php" method="POST"> <h2>Student Feedback Form</h2> <!-- Text
input --> <label for="name">Full Name:</label><br> <input type="text" id="name"
name="name" placeholder="Enter your name" required><br><br> <!-- Email input -->
<label for="email">Email:</label><br> <input type="email" id="email" name="email"
placeholder="Enter email"><br><br> <!-- Password input --> <label
for="pwd">Password:</label><br> <input type="password" id="pwd" name="pwd"><br><br>
<!-- Number input --> <label for="age">Age:</label><br> <input type="number" id="age"
name="age" min="1" max="100"><br><br> <!-- Radio buttons -->
<label>Gender:</label><br> <input type="radio" name="gender" value="male"> Male
<input type="radio" name="gender" value="female"> Female<br><br> <!-- Checkboxes -->
<label>Subjects:</label><br> <input type="checkbox" name="sub" value="html"> HTML
<input type="checkbox" name="sub" value="css"> CSS <input type="checkbox" name="sub"
value="js"> JavaScript<br><br> <!-- Dropdown select --> <label
for="year">Year:</label><br> <select id="year" name="year"> <option value="">--Select
Year--</option> <option value="1">First Year</option> <option value="2">Second
Year</option> <option value="3">Third Year</option> </select><br><br> <!-- Textarea
--> <label for="feedback">Your Feedback:</label><br> <textarea id="feedback"
name="feedback" rows="4" cols="40" placeholder="Write your feedback
here..."></textarea><br><br> <!-- Date input --> <label for="date">Date:</label>
<input type="date" id="date" name="date"><br><br> <!-- File upload --> <label
for="file">Upload File:</label> <input type="file" id="file" name="file"><br><br>
<!-- Hidden input --> <input type="hidden" name="formID" value="feedback001"> <!--
Submit and Reset buttons --> <button type="submit">Submit Feedback</button> <button
type="reset">Clear Form</button> </form>
```

Form Attributes:

- **action** — URL where form data is sent for processing.
- **method** — How data is sent: GET (visible in URL) or POST (hidden in body).
- **enctype** — Encoding: 'multipart/form-data' required for file uploads.

All Input Types:

Type	Description
------	-------------

text	Single-line text field
password	Masked text (shows ***)
email	Email field with validation
number	Numeric input with min/max
tel	Telephone number
url	URL with validation
date	Date picker
time	Time picker
radio	One choice from many (same name groups them)
checkbox	Multiple choices (check/uncheck)
file	File upload
color	Color picker
range	Slider (min to max)
hidden	Not visible to user, but sent with form
submit	Submits the form
reset	Clears all form fields
button	Generic button (no default action)
image	Image used as submit button
search	Search field

Other Form Elements:

- **<label>** — Label for an input. 'for' attribute links it to input's 'id'. Clicking label focuses the input.
- **<textarea>** — Multi-line text area. rows and cols set size.
- **<select>** — Dropdown list. Contains <option> elements.
- **<option>** — Individual item in dropdown. 'value' is what gets sent. 'selected' pre-selects it.
- **<optgroup>** — Groups options inside select.
- **<button>** — Clickable button. type can be submit, reset, or button.
- **<fieldset>** — Groups related form elements with a box.
- **<legend>** — Caption for fieldset.

Important Input Attributes:

- **name** — KEY used when form data is submitted. Required!
- **id** — Unique identifier. Links to label's 'for'.
- **value** — Default value or value submitted.
- **placeholder** — Hint text shown inside empty field.
- **required** — Field must be filled before submission.
- **disabled** — Field is shown but cannot be interacted with.
- **readonly** — Field is shown, data can be seen but not changed.
- **maxlength** — Maximum characters allowed.
- **min / max** — Minimum / maximum value (for number, date, range).

- **checked** — Pre-checks a radio or checkbox.
- **multiple** — Allows multiple file selection or select options.

EXAM: Radio buttons with same 'name' form a group — only ONE can be selected. Checkboxes can have MULTIPLE selections.

2.11 Embedding / iFrame

The <iframe> tag embeds another web page INSIDE the current page.

```
<iframe src="https://www.google.com" width="600" height="400" frameborder="0"
title="Google Map"> </iframe>
```

- **src** — URL of the page to embed.
- **width / height** — Dimensions.
- **frameborder** — 0 = no border, 1 = border.
- **scrolling** — yes, no, auto.
- **title** — Accessibility description.

Common <head> Meta Tags:

```
<meta charset="UTF-8"> <meta name="description" content="My web page about HTML">
<meta name="keywords" content="HTML, CSS, web development"> <meta name="author"
content="John Doe"> <meta name="viewport" content="width=device-width,
initial-scale=1.0"> <meta http-equiv="refresh" content="30"> <!-- Refreshes page
every 30 sec -->
```

UNIT III — CASCADING STYLE SHEETS (CSS)

3.1 What is CSS?

CSS stands for **Cascading Style Sheets**. It is used to control the APPEARANCE and LAYOUT of HTML elements — colors, fonts, spacing, positioning, sizes, animations, and more.

- **Cascading** = Styles can come from multiple sources; there is an order of priority.
- **Style** = Visual presentation rules.
- **Sheets** = Saved in .css files.

NOTE: HTML structures the content. CSS styles (beautifies) the content. JavaScript makes it interactive.

3.2 CSS Syntax

```
selector { property: value; property: value; }
```

- **Selector** — Targets which HTML element(s) to style.
- **Property** — What to style (e.g., color, font-size).
- **Value** — How to style it (e.g., red, 16px).
- **Declaration** — One property:value pair.
- **Declaration Block** — All declarations inside { }.

```
/* This is a CSS Comment */ p { color: blue; /* Text color */ font-size: 16px; /* Font size */ }
```

EXAM: CSS comments use `/* ... */` NOT `//` like in JavaScript.

3.3 Types of CSS

1. Inline CSS:

Written directly inside the HTML tag using the style attribute.

```
<p style="color: red; font-size: 20px;">This text is red.</p>
```

- Highest specificity (priority) — overrides other CSS.
- Hard to maintain — not recommended for large projects.

2. Internal CSS (Embedded):

Written inside a <style> tag in the <head> section.

```
<head> <style> p { color: blue; font-size: 18px; } h1 { color: green; } </style> </head>
```

- Applies to the entire page.
- Good for single-page styling.

3. External CSS:

Written in a separate .css file, linked using <link> in <head>.

```
<!-- In HTML file (index.html) --> <head> <link rel="stylesheet" href="styles.css">
</head> /* In CSS file (styles.css) */ p { color: green; }
```

- Best practice for large projects.
- One CSS file can style many HTML pages.
- Easy to maintain.

EXAM: Priority / Specificity order: Inline > Internal > External. Inline CSS ALWAYS wins unless !important is used.

3.4 CSS Selectors

```
/* Universal Selector – selects ALL elements */ * { margin: 0; padding: 0; } /*
Element/Type Selector – selects by tag name */ p { color: black; } h1 { font-size:
32px; } /* Class Selector – selects by class attribute (.classname) */ .highlight {
background-color: yellow; } /* Used in HTML: <p class="highlight">Text</p> */ /* ID
Selector – selects by id attribute (#idname) */ #header { background-color: navy; }
/* Used in HTML: <div id="header">...</div> */ /* Descendant Selector – p inside div
*/ div p { color: red; } /* Child Selector – direct child only */ div > p { color:
blue; } /* Adjacent Sibling – immediately after */ h1 + p { font-weight: bold; } /*
General Sibling */ h1 ~ p { color: gray; } /* Attribute Selector */
input[type="text"] { border: 1px solid blue; } a[href="https://google.com"] { color:
red; } /* Pseudo-class Selectors */ a:hover { color: orange; } /* Mouse over */
a:visited { color: purple; } /* Already visited */ a:active { color: red; } /* Being
clicked */ a:link { color: blue; } /* Unvisited */ p:first-child { font-weight: bold;
} p:last-child { color: gray; } p:nth-child(2) { color: green; } /* 2nd child */ /*
Pseudo-element Selectors */ p::first-line { font-weight: bold; } p::first-letter {
font-size: 200%; } p::before { content: ">> "; } p::after { content: " <<"; }
```

EXAM: ID is unique (one per page). Class can be reused multiple times. ID uses #, class uses .

3.5 Colors in CSS

```
color: red; /* Named color */ color: #ff0000; /* Hex color */ color: #f00; /*
Shorthand hex */ color: rgb(255, 0, 0); /* RGB */ color: rgba(255, 0, 0, 0.5); /* RGB
+ Alpha (opacity 0-1) */ color: hsl(0, 100%, 50%); /* Hue, Saturation, Lightness */
color: hsla(0, 100%, 50%, 0.5); /* HSL + Alpha */
```

- Hex: #RRGGBB. #000000 = black. #ffffff = white. #ff0000 = red.
- RGB: rgb(red, green, blue) — each 0 to 255.
- RGBA: 4th value is opacity — 0 = transparent, 1 = opaque.

3.6 Text and Font Properties

```
/* FONT PROPERTIES */ font-family: Arial, Helvetica, sans-serif; /* Font name,
fallbacks */ font-size: 16px; /* Size in px, em, rem, % */ font-weight: bold; /*
bold, normal, 100-900 */ font-style: italic; /* normal, italic, oblique */
font-variant: small-caps; /* TEXT PROPERTIES */ color: #333; /* Text color */
text-align: center; /* left, right, center, justify */ text-decoration: underline; /*
none, underline, overline, line-through */ text-transform: uppercase; /* uppercase,
lowercase, capitalize */ text-indent: 30px; /* First line indent */ letter-spacing:
2px; /* Space between characters */ word-spacing: 5px; /* Space between words */
line-height: 1.6; /* Space between lines */ text-shadow: 2px 2px 4px gray; /*
```

```
H-offset V-offset blur color */
```

3.7 Background Properties

```
background-color: #f0f0f0; background-image: url("bg.jpg"); background-repeat:
no-repeat; /* repeat, repeat-x, repeat-y, no-repeat */ background-position: center
top; background-size: cover; /* cover, contain, auto, 100px 200px */
background-attachment: fixed; /* scroll, fixed */ /* Shorthand */ background: #f0f0f0
url("bg.jpg") no-repeat center center / cover;
```

3.8 The CSS Box Model

Every HTML element is a rectangular box. The box model defines the space an element takes up.

```
/* From inside to outside: */ /* CONTENT → PADDING → BORDER → MARGIN */ div { width:
300px; /* Width of content area */ height: 200px; /* Height of content area */
padding: 20px; /* Space INSIDE, between content and border */ padding-top: 10px;
padding-right: 20px; padding-bottom: 10px; padding-left: 20px; border: 2px solid
black; /* Border around the element */ border-width: 2px; border-style: solid; /*
solid, dashed, dotted, double, none */ border-color: black; border-radius: 10px; /*
Rounded corners */ margin: 30px; /* Space OUTSIDE, between elements */ margin: 10px
20px; /* Top-bottom=10, Left-right=20 */ margin: 10px 20px 30px 40px; /* Top Right
Bottom Left (clockwise) */ margin: auto; /* Centre the element horizontally */ } /*
box-sizing changes what width/height includes */ box-sizing: content-box; /* Default:
width = content only */ box-sizing: border-box; /* Width includes padding + border
(preferred!) */
```

EXAM: PADDING = space INSIDE the border. MARGIN = space OUTSIDE the border. Padding adds to background color. Margin is transparent.

3.9 Display and Position Properties

```
/* DISPLAY */ display: block; /* Takes full width, starts new line. (div, p, h1) */
display: inline; /* Takes only needed width, same line. (span, a) */ display:
inline-block; /* Inline but respects width/height. */ display: none; /* Completely
hides element (removed from layout). */ display: flex; /* Flexbox container */
display: grid; /* CSS Grid container */ /* POSITION */ position: static; /* Default.
Normal flow. top/left/right/bottom ignored. */ position: relative; /* Moves relative
to its NORMAL position. */ position: absolute; /* Removed from flow. Positioned
relative to nearest positioned ancestor. */ position: fixed; /* Stays fixed relative
to VIEWPORT (doesn't scroll). */ position: sticky; /* Stays in flow until scroll
threshold, then fixed. */ /* Use with: */ top: 10px; right: 20px; bottom: 0; left:
50px; z-index: 100; /* Stack order. Higher = on top. */
```

EXAM: position:fixed stays on screen even when scrolling (navbar). position:absolute is relative to parent. z-index controls which element appears on top.

3.10 Flexbox

```
/* PARENT (container) */ .container { display: flex; flex-direction: row; /* row
(default), row-reverse, column, column-reverse */ flex-wrap: wrap; /* nowrap, wrap,
wrap-reverse */ justify-content: center; /* Align items on MAIN axis */ /*
flex-start, flex-end, center, space-between, space-around, space-evenly */
align-items: center; /* Align items on CROSS axis */ /* flex-start, flex-end, center,
```



```
stretch, baseline */ gap: 10px; /* Space between items */ } /* CHILD (item) */ .item
{ flex-grow: 1; /* How much the item grows relative to others */ flex-shrink: 1; /*
How much it shrinks */ flex-basis: auto; /* Initial size before growing/shrinking */
flex: 1; /* Shorthand: grow shrink basis */ order: 2; /* Display order (default 0) */
align-self: flex-end; /* Override container's align-items for this item */ }
```

EXAM: justify-content aligns on the MAIN axis (horizontal by default). align-items aligns on the CROSS axis (vertical by default).

3.11 Common CSS Units

Unit	Type	Description / Example
px	Absolute	Pixels — fixed size. 16px
em	Relative	Relative to PARENT font size. 2em = 2× parent
rem	Relative	Relative to ROOT font size. More consistent than em
%	Relative	Percentage of parent element. width: 50%
vw	Viewport	1% of viewport WIDTH. 100vw = full screen width
vh	Viewport	1% of viewport HEIGHT. 100vh = full screen height
pt	Absolute	Points. Mainly for print stylesheets. 1pt=1/72 inch

Other Important CSS Properties:

```
/* OVERFLOW — what happens when content is too big */ overflow: visible; /* Default:
content spills out */ overflow: hidden; /* Extra content is clipped/hidden */
overflow: scroll; /* Always shows scrollbar */ overflow: auto; /* Scrollbar only if
needed */ /* CURSOR */ cursor: pointer; /* Hand cursor (like hovering a link) */
cursor: default; /* Normal arrow */ cursor: text; /* Text I-beam */ /* OPACITY */
opacity: 0.5; /* 0 = invisible, 1 = fully visible */ /* TRANSITION — smooth animation
between state changes */ transition: all 0.3s ease; transition: color 0.5s,
background 1s; /* BOX SHADOW */ box-shadow: 5px 5px 10px rgba(0,0,0,0.3); /* h-offset
v-offset blur spread color */ /* VISIBILITY */ visibility: hidden; /* Hides element
but keeps space */ display: none; /* Hides element AND removes space */
```

UNIT IV — JAVASCRIPT AND DOM

4.1 What is JavaScript?

JavaScript (JS) is a lightweight, interpreted programming language that runs in the browser. It makes web pages INTERACTIVE and DYNAMIC.

- Created by Brendan Eich in 1995 (originally called Mocha, then LiveScript).
- Runs on the CLIENT SIDE (in the browser) — no server needed.
- Also used server-side with Node.js.
- Can modify HTML content, respond to user events, validate forms, fetch data, and much more.

NOTE: JavaScript is NOT related to Java. They are completely different languages!

Adding JavaScript to HTML:

```
<!-- Method 1: Inline (inside tag – bad practice) --> <button
onclick="alert('Clicked!')">Click Me</button> <!-- Method 2: Internal (inside
<script> tag) --> <script> alert("Hello from internal script!"); </script> <!--
Method 3: External file (best practice) --> <script src="script.js"></script> <!--
Place scripts at bottom of body for better performance --> <body> ...content...
<script src="script.js"></script> </body>
```

4.2 Variables and Data Types

```
// Three ways to declare variables: var name = "Alice"; // Old way. Function-scoped.
Can be re-declared. let age = 20; // Modern. Block-scoped. Cannot be re-declared.
const PI = 3.14159; // Constant. Cannot be changed after assignment. // Data Types:
let str = "Hello"; // String let num = 42; // Number (integer or decimal) let dec =
3.14; // Number (float) let bool = true; // Boolean (true or false) let nothing =
null; // Null – intentionally empty let undef; // Undefined – declared but not
assigned let arr = [1, 2, 3]; // Array let obj = {name: "Bob"}; // Object // Check
data type: typeof "Hello" // "string" typeof 42 // "number" typeof true // "boolean"
typeof undefined // "undefined" typeof null // "object" (historical bug in JS!)
typeof [] // "object" typeof {} // "object"
```

EXAM: var is function-scoped. let and const are block-scoped. const cannot be reassigned. let and const are preferred in modern JS.

String Operations:

```
let firstName = "John"; let lastName = "Doe"; // Concatenation (joining strings) let
full = firstName + " " + lastName; // "John Doe" // Template Literals (backticks –
modern way) let greeting = `Hello, ${firstName}!`; // "Hello, John!" let multi =
`Line 1
Line 2`; // Multi-line string // String Methods: "hello".length // 5
"hello".toUpperCase() // "HELLO" "HELLO".toLowerCase() // "hello" "hello
world".indexOf("world") // 6 "hello".includes("ell") // true "hello".slice(1, 3) //
"el" "hello".replace("hello", "hi") // "hi" "hello ".trim() // "hello"
"a,b,c".split(",") // ["a", "b", "c"]
```

4.3 Operators

```
// ARITHMETIC OPERATORS 5 + 3 // 8 (addition) 5 - 3 // 2 (subtraction) 5 * 3 // 15
(multiplication) 10 / 4 // 2.5 (division) 10 % 3 // 1 (modulus – remainder) 2 ** 3 //
8 (exponent – 2 to the power 3) x++ // Increment by 1 (post-increment) ++x //
Increment by 1 (pre-increment) x-- // Decrement by 1 // ASSIGNMENT OPERATORS x = 5; x
+= 2; x -= 1; x *= 3; x /= 2; x %= 2; // COMPARISON OPERATORS (return true or false) 5
== "5" // true (loose: checks value only) 5 === "5" // false (strict: checks value
AND type) 5 != "5" // false (loose not-equal) 5 !== "5" // true (strict not-equal) 5 >
3 // true 5 < 3 // false 5 >= 5 // true 5 <= 4 // false // LOGICAL OPERATORS true &&
false // false (AND – both must be true) true || false // true (OR – at least one must
be true) !true // false (NOT – reverses) // TERNARY OPERATOR let result = (age >= 18)
? "Adult" : "Minor";
```

EXAM: === is STRICT equality (checks type too). == is loose equality. Always use === in JavaScript to avoid bugs!

4.4 Decision Making — If / Else / Switch

```
// IF / ELSE IF / ELSE let marks = 75; if (marks >= 90) { console.log("Grade: A"); }
else if (marks >= 75) { console.log("Grade: B"); } else if (marks >= 60) {
console.log("Grade: C"); } else { console.log("Grade: Fail"); } // SWITCH STATEMENT
let day = "Monday"; switch (day) { case "Monday": console.log("Start of week");
break; case "Friday": console.log("End of week"); break; case "Saturday": case
"Sunday": console.log("Weekend!"); break; default: console.log("Midweek"); }
```

EXAM: break is MANDATORY in switch cases. Without break, execution 'falls through' to the next case!

4.5 Loops

```
// FOR LOOP – when you know the count for (let i = 0; i < 5; i++) {
console.log("Count: " + i); // 0,1,2,3,4 } // WHILE LOOP – runs while condition is
true let i = 0; while (i < 5) { console.log(i); i++; } // DO-WHILE LOOP – runs AT
LEAST ONCE let n = 0; do { console.log(n); n++; } while (n < 3); // FOR...OF – iterate
over array values let fruits = ["Apple", "Banana", "Mango"]; for (let fruit of
fruits) { console.log(fruit); } // FOR...IN – iterate over object keys let person =
{name: "Alice", age: 20}; for (let key in person) { console.log(key + ": " +
person[key]); } // BREAK – exits loop immediately // CONTINUE – skips current
iteration, goes to next
```

4.6 Arrays

```
// Creating Arrays let fruits = ["Apple", "Banana", "Mango"]; let mixed = [1,
"Hello", true, null]; let empty = []; let matrix = [[1,2],[3,4]]; // 2D array //
Accessing Elements (0-indexed) fruits[0] // "Apple" fruits[2] // "Mango"
fruits[fruits.length - 1] // Last element // Array Methods fruits.push("Orange") //
Adds to END. Returns new length. fruits.pop() // Removes from END. Returns removed
element. fruits.unshift("Grape") // Adds to START. Returns new length. fruits.shift()
// Removes from START. Returns removed element. fruits.length // Number of elements
fruits.indexOf("Banana") // 1 (index), -1 if not found fruits.includes("Mango") //
true fruits.join(", ") // "Apple, Banana, Mango" fruits.reverse() // Reverses in
place fruits.sort() // Sorts alphabetically fruits.slice(1, 3) // Returns NEW array
[Banana, Mango] (doesn't modify) fruits.splice(1, 1) // Removes 1 element at index 1
(modifies original) fruits.splice(1, 0, "Kiwi") // Inserts "Kiwi" at index 1
fruits.forEach(function(item) { console.log(item); }); let upper = fruits.map(f =>
```

```
f.toUpperCase()); let long = fruits.filter(f => f.length > 5); let found = fruits.find(f => f === "Mango");
```

4.7 Functions

```
// FUNCTION DECLARATION function greet(name) { return "Hello, " + name + "!"; }  
greet("Alice"); // "Hello, Alice!" // FUNCTION EXPRESSION const add = function(a, b)  
{ return a + b; }; add(3, 4); // 7 // ARROW FUNCTION (ES6 – modern, shorter syntax)  
const multiply = (a, b) => a * b; const square = x => x * x; // Single param – no ()  
const sayHi = () => "Hello!"; // No params // DEFAULT PARAMETERS function greet(name  
= "World") { return `Hello, ${name}!`; } greet(); // "Hello, World!" greet("Bob"); //  
"Hello, Bob!" // REST PARAMETERS (accepts any number of args) function  
sum(...numbers) { return numbers.reduce((a, b) => a + b, 0); } sum(1, 2, 3, 4); // 10  
// ANONYMOUS FUNCTION (no name) setTimeout(function() { alert("3 seconds passed!");  
}, 3000); // IIFE – Immediately Invoked Function Expression (function() {  
console.log("Runs immediately!"); })(); // Function scope – variables inside function  
are local function test() { let x = 10; // x is only accessible inside test() }
```

EXAM: return stops the function and sends back a value. If no return, function returns undefined.

4.8 Objects

```
// Creating an Object let student = { name: "Alice", age: 20, grade: "A", greet:  
function() { return "Hi, I'm " + this.name; } }; // Accessing properties student.name  
// Dot notation – "Alice" student["name"] // Bracket notation – "Alice"  
student.greet() // Calling method – "Hi, I'm Alice" // Adding / Modifying properties  
student.city = "Mumbai"; // Add new property student.age = 21; // Modify existing //  
Delete property delete student.grade; // this keyword – refers to the current object  
// Checking property existence "name" in student // true
```

4.9 DOM — Document Object Model

The DOM is a programming interface for HTML documents. When a browser loads a page, it creates a DOM tree — a hierarchical representation of all elements. JavaScript can access and modify this tree to change the page dynamically.

DOM Structure:

```
Document ■■■ html ■■■ head ■■■ title ■■■ body ■■■ h1 ■■■ p ■■■ div ■■■ p
```

Selecting DOM Elements:

```
// Get single element by ID let el = document.getElementById("myId"); // Get elements  
by class name (HTMLCollection) let els = document.getElementsByClassName("myClass");  
// Get elements by tag name (HTMLCollection) let paras =  
document.getElementsByTagName("p"); // Modern – CSS selector (returns FIRST match)  
let el = document.querySelector("#myId"); // By ID let el =  
document.querySelector(".myClass"); // By class let el = document.querySelector("p");  
// By tag let el = document.querySelector("div > p"); // By CSS selector // Modern –  
CSS selector (returns ALL matches as NodeList) let all =  
document.querySelectorAll(".myClass"); let all = document.querySelectorAll("p");
```

Modifying DOM Elements:

```
let el = document.getElementById("demo"); // Change content el.innerHTML = "<b>Bold text</b>"; // Parses HTML tags el.textContent = "Plain text"; // Treats everything as text (safer) el.innerText = "Visible text"; // Similar to textContent // Change style el.style.color = "red"; el.style.backgroundColor = "#fff"; el.style.fontSize = "20px"; // Change attributes el.setAttribute("class", "newClass"); el.getAttribute("href"); el.removeAttribute("disabled"); // Change class el.className = "myClass"; // Replaces all classes el.classList.add("active"); // Add class el.classList.remove("active"); // Remove class el.classList.toggle("active"); // Toggle on/off el.classList.contains("active"); // Check if has class
```

Creating and Removing Elements:

```
// Create new element let newPara = document.createElement("p"); newPara.textContent = "New paragraph"; newPara.style.color = "blue"; // Add to DOM document.body.appendChild(newPara); // At end of body document.getElementById("div1").appendChild(newPara); // Insert before let parent = document.getElementById("list"); let ref = document.getElementById("item2"); parent.insertBefore(newPara, ref); // Remove element el.parentNode.removeChild(el); // Old way el.remove(); // Modern way
```

4.10 Events and Event Handlers

```
// Method 1: HTML attribute (inline – avoid for large projects) <button onclick="myFunction()">Click Me</button> // Method 2: DOM property document.getElementById("btn").onclick = function() { alert("Button clicked!"); }; // Method 3: addEventListener (BEST practice – allows multiple listeners) document.getElementById("btn").addEventListener("click", function() { alert("Clicked!"); }); // Common Events: // Mouse: click, dblclick, mouseover, mouseout, mouseenter, mouseleave, mousemove // Keyboard: keydown, keyup, keypress // Form: submit, change, input, focus, blur, reset // Window: load, resize, scroll, unload // Touch: touchstart, touchend, touchmove // Event Object document.getElementById("btn").addEventListener("click", function(event) { console.log(event.type); // "click" console.log(event.target); // The element that was clicked event.preventDefault(); // Stop default behavior (e.g., form submit) event.stopPropagation(); // Stop event from bubbling up }); // CASE STUDY: Show popup (alert) for confirmation function confirmDelete() { let result = confirm("Are you sure you want to delete?"); if (result) { alert("Item deleted!"); } else { alert("Cancelled."); } }
```

Built-in Dialog Boxes:

- **alert('message')** — Shows a message popup. User clicks OK.
- **confirm('question')** — Shows OK / Cancel. Returns true or false.
- **prompt('question', 'default')** — Gets text input from user. Returns string or null.

EXAM: alert() just shows a message. confirm() returns true (OK) or false (Cancel). prompt() returns the typed text.

4.11 JavaScript Prototypes

Every JavaScript object has a prototype — a template object from which it inherits methods and properties. Prototypes enable inheritance in JavaScript.

```
// Constructor Function
function Person(name, age) { this.name = name; this.age = age; }
// Adding method via prototype (shared by all instances – memory efficient)
Person.prototype.greet = function() { return "Hi, I'm " + this.name; };
let p1 = new Person("Alice", 20);
let p2 = new Person("Bob", 25);
p1.greet(); // "Hi, I'm Alice"
p2.greet(); // "Hi, I'm Bob"
// Modern alternative – ES6 Classes
class Animal {
  constructor(name) { this.name = name; }
  speak() { return this.name + " makes a sound."; }
}
class Dog extends Animal {
  speak() { return this.name + " barks!"; }
}
let d = new Dog("Rex");
d.speak(); // "Rex barks!"
// Prototype chain: d → Dog.prototype → Animal.prototype → Object.prototype
```

UNIT V — BACK-END TECHNOLOGIES (PHP)

5.1 What is PHP?

PHP stands for **PHP: Hypertext Preprocessor** (recursive acronym). It is a widely-used server-side scripting language designed for web development.

- Created by Rasmus Lerdorf in 1994.
- Runs on the SERVER — the browser never sees PHP code, only the output (HTML).
- Embedded directly inside HTML.
- Free, open-source.
- Connects to databases (MySQL) to create dynamic web pages.
- File extension: **.php**

NOTE: PHP code runs on the server. JavaScript runs in the browser. PHP sends HTML to browser.

How PHP Works:

1. Browser requests a .php page.
2. Web server (Apache/Nginx) passes it to PHP interpreter.
3. PHP executes the code, queries database if needed.
4. PHP produces HTML output.
5. Server sends HTML back to browser.

5.2 PHP Document Structure

```
<?php // This is a PHP comment (single line) /* This is a multi-line comment */ echo "Hello, World!"; // Output: Hello, World! ?>
```

```
<!-- PHP inside HTML --> <!DOCTYPE html> <html> <head><title>PHP Page</title></head> <body> <h1><?php echo "Hello from PHP!"; ?></h1> <p>Today is: <?php echo date("d/m/Y"); ?></p> </body> </html>
```

- **<?php** — Opens a PHP block. All PHP code goes between this and ?>
- **?>** — Closes the PHP block.
- **echo** — Outputs data to the browser.
- **print** — Similar to echo but slightly different (returns 1, can only output one string).
- Every PHP statement ends with a **semicolon ;**

EXAM: PHP tags: <?php ... ?> — always use the full opening tag, not short tag <? ?>

5.3 Variables and Data Types

```
<?php // Variables start with $ sign $name = "Alice"; // String $age = 20; // Integer $price = 19.99; // Float/Double $isValid = true; // Boolean (true/false) $nothing = null; // Null $arr = [1, 2, 3]; // Array // Variable names: // - Must start with $ followed by letter or underscore // - Case-sensitive: $name and $Name are different! // - Cannot start with a number // PHP is loosely typed – no need to declare type $x = 10; // integer $x = "ten"; // now a string – same variable // Constants (no $ sign, cannot be changed) define("PI", 3.14159); const MAX_SIZE = 100; echo PI; // 3.14159
```

```
// Variable variables $varName = "city"; $$varName = "Mumbai"; echo $city; //
"Mumbai" // Check variable type gettype($age); // "integer" is_int($age); // true
is_string($name); // true is_null($nothing); // true // Type casting $x = "42"; $y =
(int)$x; // 42 (integer) $z = (float)$x; // 42.0
```

EXAM: All PHP variables start with \$. PHP is case-sensitive for variable names!

5.4 Operators

```
<?php // ARITHMETIC $a + $b // Addition $a - $b // Subtraction $a * $b //
Multiplication $a / $b // Division $a % $b // Modulus (remainder) $a ** $b //
Exponentiation // ASSIGNMENT $x = 5; $x += 3; $x -= 2; $x *= 4; $x /= 2; $x %= 3; //
COMPARISON $a == $b // Equal (value only) $a === $b // Identical (value AND type) $a
!= $b // Not equal $a !== $b // Not identical $a > $b // Greater than $a < $b // Less
than $a >= $b // Greater or equal $a <= $b // Less or equal $a <=> $b // Spaceship:
returns -1, 0, or 1 // LOGICAL $a && $b // AND $a || $b // OR !$a // NOT $a and $b //
AND (lower precedence) $a or $b // OR (lower precedence) // STRING $a . $b //
Concatenation $a .= $b // Concatenate and assign // INCREMENT/DECREMENT $x++ $x--
++$x --$x // NULL COALESCING $val = $x ?? "default"; // Use $x if set, else "default"
?>
```

5.5 String Functions

```
<?php $str = "Hello, World!"; strlen($str) // 13 – length of string strtolower($str)
// "hello, world!" strtoupper($str) // "HELLO, WORLD!" ucfirst($str) // "Hello,
world!" – first letter uppercase ucwords("hello world") // "Hello World" – each word
capitalised trim(" hello ") // "hello" – removes leading/trailing whitespace ltrim("
hello") // "hello" – left trim rtrim("hello ") // "hello" – right trim
str_replace("World","PHP",$str) // "Hello, PHP!" str_repeat("ab", 3) // "ababab"
str_word_count("Hello World") // 2 strpos($str, "World") // 7 – position of first
occurrence strrpos($str, "l") // 10 – position of last occurrence substr($str, 7, 5)
// "World" – substring from pos 7, length 5 strrev("Hello") // "olleH" – reverse
string strcmp("a","b") // negative (a < b) strcasecmp("Hello","hello") // 0 (equal,
case-insensitive) // Splitting and joining explode(", ", "a,b,c") // ["a","b","c"] –
split string to array implode("-", ["a","b","c"]) // "a-b-c" – join array to string
// Checking str_contains($str, "World") // true – PHP 8+
str_starts_with($str,"Hello") // true – PHP 8+ str_ends_with($str,"!") // true – PHP
8+ // Formatting number_format(1234567.891, 2) // "1,234,567.89" sprintf("%.2f",
3.14159) // "3.14" printf("Name: %s, Age: %d", "Alice", 20); // prints formatted //
HTML htmlspecialchars("<b>Bold</b>") // "&lt;b&gt;Bold&lt;/b&gt;" – security!
nl2br("Hello World") // "Hello<br>World" ?>
```

EXAM: htmlspecialchars() is VERY important — converts < > & ' " to safe HTML entities. Prevents XSS attacks!

5.6 Decision Making

```
<?php $score = 75; // IF / ELSEIF / ELSE if ($score >= 90) { echo "Grade: A"; } elseif
($score >= 75) { // NOTE: elseif (not else if) is one word in PHP echo "Grade: B"; }
elseif ($score >= 60) { echo "Grade: C"; } else { echo "Fail"; } // SWITCH $day =
"Monday"; switch ($day) { case "Monday": echo "Week starts"; break; case "Friday":
echo "TGIF!"; break; case "Saturday": case "Sunday": echo "Weekend"; break; default:
echo "Midweek"; } // TERNARY OPERATOR $status = ($age >= 18) ? "Adult" : "Minor"; //
NULL COALESCING $name = $_GET["name"] ?? "Guest"; // MATCH EXPRESSION (PHP 8+)
```



```
$result = match($score) { 90, 95, 100 => "A", 75, 80, 85 => "B", default => "C" }; ?>
```

5.7 Loops

```
<?php // FOR LOOP for ($i = 0; $i < 5; $i++) { echo $i . " "; // 0 1 2 3 4 } // WHILE  
LOOP $i = 1; while ($i <= 5) { echo $i . " "; $i++; } // DO-WHILE (executes at least  
once) $i = 1; do { echo $i . " "; $i++; } while ($i <= 5); // FOREACH – best for  
arrays $fruits = ["Apple", "Banana", "Mango"]; foreach ($fruits as $fruit) { echo  
$fruit . "<br>"; } // FOREACH with key => value $person = ["name" => "Alice", "age"  
=> 20]; foreach ($person as $key => $value) { echo "$key: $value<br>"; } // BREAK and  
CONTINUE for ($i = 0; $i < 10; $i++) { if ($i == 5) break; // Exit loop when i=5 if  
($i % 2 == 0) continue; // Skip even numbers echo $i . " "; // 1 3 } ?>
```

EXAM: foreach is the most common loop for PHP arrays. Use for when you know the count, while when condition-based.

5.8 PHP Functions

```
<?php // Basic Function function greet($name) { return "Hello, " . $name . "!"; }  
echo greet("Alice"); // Hello, Alice! // Default Parameter function greetUser($name =  
"World") { return "Hello, " . $name . "!"; } greetUser(); // Hello, World!  
greetUser("Bob"); // Hello, Bob! // Return Multiple Values (using array) function  
minMax($arr) { return [min($arr), max($arr)]; } [$min, $max] = minMax([3, 1, 5, 2]);  
// Variable-length arguments function sum(...$nums) { return array_sum($nums); } echo  
sum(1, 2, 3, 4); // 10 // Pass by Reference (&) function increment(&$val) { $val++; }  
$x = 5; increment($x); echo $x; // 6 (original changed!) // Anonymous Function  
$square = function($n) { return $n * $n; }; echo $square(4); // 16 // Recursive  
Function function factorial($n) { if ($n <= 1) return 1; return $n * factorial($n -  
1); } echo factorial(5); // 120 // Variable functions function sayHello() { echo  
"Hello!"; } $func = "sayHello"; $func(); // calls sayHello() ?>
```

5.9 Arrays

```
<?php // INDEXED ARRAY $fruits = ["Apple", "Banana", "Mango"]; $nums = array(1, 2, 3,  
4, 5); echo $fruits[0]; // Apple echo $fruits[2]; // Mango $fruits[] = "Orange"; //  
Append to end // ASSOCIATIVE ARRAY (key => value pairs) $student = [ "name" =>  
"Alice", "age" => 20, "grade" => "A" ]; echo $student["name"]; // Alice  
$student["city"] = "Mumbai"; // Add new key // MULTIDIMENSIONAL ARRAY $students = [  
["name" => "Alice", "marks" => 90], ["name" => "Bob", "marks" => 85], ]; echo  
$students[0]["name"]; // Alice echo $students[1]["marks"]; // 85 // ARRAY FUNCTIONS  
count($fruits) // 3 – number of elements array_push($fruits, "Grape") // Add to end  
array_pop($fruits) // Remove from end, return it array_shift($fruits) // Remove from  
start, return it array_unshift($fruits, "Kiwi") // Add to start array_reverse($fruits)  
// Reverse order (returns new array) sort($fruits) // Sort indexed array ascending  
rsort($fruits) // Sort descending asort($student) // Sort associative by value  
ksort($student) // Sort associative by key arsort($student) // Sort associative by  
value descending krsort($student) // Sort associative by key descending  
in_array("Apple", $fruits) // true – check if value exists array_key_exists("name",  
$student) // true array_search("Banana", $fruits) // 1 – returns key of value  
array_merge($arr1, $arr2) // Merge two arrays array_slice($arr, 1, 3) // Extract  
portion (from, count) array_splice($arr, 1, 2) // Remove and replace elements  
array_unique($arr) // Remove duplicates array_flip($arr) // Swap keys and values  
array_combine($keys, $vals) // Create array from keys+values arrays  
array_map(function($x){ return $x*2; }, $nums) // Apply function to each
```

```

array_filter($nums, function($x){ return $x>2; });// Filter elements
array_reduce($nums, function($carry,$x){ return $carry+$x; }, 0) // Reduce implode(",
", $fruits) // "Apple, Banana, Mango" (array to string) explode(",", "a,b,c") //
["a","b","c"] (string to array) print_r($fruits); // Human-readable dump
var_dump($fruits); // Detailed dump with types ?>

```

5.10 PHP Superglobals and Form Handling

Superglobals are built-in PHP variables accessible anywhere. Case Study: Print submitted data as confirmation.

```

<?php // $_GET – receives data from URL (GET method) // URL:
page.php?name=Alice&age=20 $name = $_GET["name"]; // "Alice" $age = $_GET["age"]; //
"20" // $_POST – receives data from form (POST method – hidden in body) $name =
$_POST["name"]; // $_REQUEST – contains both GET and POST data // Safe way (avoid
errors if key doesn't exist): $name = isset($_POST["name"]) ? $_POST["name"] : ""; //
OR (PHP 7+): $name = $_POST["name"] ?? ""; // CASE STUDY: Process feedback form if
($_SERVER["REQUEST_METHOD"] == "POST") { $name = htmlspecialchars($_POST["name"] ??
""); $email = htmlspecialchars($_POST["email"] ?? ""); $feedback =
htmlspecialchars($_POST["feedback"] ?? ""); echo "<h2>Thank you, $name!</h2>"; echo
"<p>Your email: $email</p>"; echo "<p>Your feedback: $feedback</p>"; // Optional:
Save to file $data = "Name: $name | Email: $email | Feedback: $feedback ";
file_put_contents("feedback.txt", $data, FILE_APPEND); } // OTHER SUPERGLOBALS: //
$_SERVER – server and request info $_SERVER["PHP_SELF"] // Current script name
$_SERVER["REQUEST_METHOD"] // "GET" or "POST" $_SERVER["SERVER_NAME"] //
"www.example.com" $_SERVER["HTTP_HOST"] // Host header $_SERVER["REMOTE_ADDR"] //
Client IP address // $_SESSION – store data across pages session_start(); // Must be
at top of EVERY page! $_SESSION["username"] = "Alice"; // Store echo
$_SESSION["username"]; // Retrieve unset($_SESSION["username"]); // Delete one
session_destroy(); // Delete all sessions // $_COOKIE – small data stored in browser
setcookie("user", "Alice", time()+3600); // Set (expires in 1 hour) echo
$_COOKIE["user"]; // Read setcookie("user", "", time()-3600); // Delete (set past
expiry) // $_FILES – file uploads if (isset($_FILES["myfile"])) { $name =
$_FILES["myfile"]["name"]; $tmp_name = $_FILES["myfile"]["tmp_name"]; $size =
$_FILES["myfile"]["size"]; $type = $_FILES["myfile"]["type"]; $error =
$_FILES["myfile"]["error"]; move_uploaded_file($tmp_name, "uploads/" . $name); } ?>

```

EXAM: Always use htmlspecialchars() when displaying user input to prevent XSS (Cross-Site Scripting) attacks!

5.11 PHP Math and Date Functions

```

<?php // MATH FUNCTIONS abs(-5) // 5 – absolute value ceil(4.2) // 5 – round UP
floor(4.9) // 4 – round DOWN round(4.5) // 5 – round to nearest round(3.14159, 2) //
3.14 – round to 2 decimal places sqrt(16) // 4 – square root pow(2, 8) // 256 – power
max(3, 7, 1) // 7 – maximum value min(3, 7, 1) // 1 – minimum value rand(1, 100) //
Random number between 1 and 100 pi() // 3.14159... intval(10, 3) // 3 – integer
division // DATE FUNCTIONS date("Y") // "2024" – current year date("m") // "05" –
current month (with leading zero) date("d") // "15" – current day date("H:i:s") //
"14:30:00" – time date("D") // "Mon" – day name short date("l") // "Monday" – day
name full date("d/m/Y") // "15/05/2024" date("d-M-Y H:i") // "15-May-2024 14:30"
time() // Unix timestamp (seconds since 1970) mktime(0,0,0,12,25,2024) // Timestamp
for Dec 25, 2024 strtotime("next Monday") // Timestamp for next Monday date("Y",
strtotime("+1 year")) // Next year // Common date format characters: // Y=4-digit

```

```
year, y=2-digit year // m=month(01-12), M=month name(Jan), n=month(1-12) //  
d=day(01-31), D=day name(Mon), l=day name(Monday) // H=24hr hour, h=12hr hour,  
i=minutes, s=seconds // A=AM/PM, a=am/pm ?>
```

5.12 PHP File Handling

```
<?php // Writing to a file file_put_contents("data.txt", "Hello World"); // Overwrite  
file_put_contents("data.txt", "More ", FILE_APPEND); // Append // Reading from a file  
$content = file_get_contents("data.txt"); $lines = file("data.txt"); // Returns array  
of lines // Manual file operations $file = fopen("data.txt", "r"); // Open (r=read,  
w=write, a=append) while (!feof($file)) { $line = fgets($file); echo $line; }  
fclose($file); // File info file_exists("data.txt") // true/false  
filesize("data.txt") // bytes is_file("data.txt") // true is_dir("uploads") // true  
if directory // Create directory mkdir("newFolder"); ?>
```

QUICK REFERENCE — KEY CONCEPTS FOR ORAL EXAM

Topic	Key Point
<!DOCTYPE html>	NOT a tag. Document type declaration. Must be first line. Tells browser it's HTML5.
HTTP vs HTTPS	HTTP = unsecured (port 80). HTTPS = secured with SSL (port 443).
GET vs POST	GET: data in URL (visible). POST: data in body (hidden). POST for sensitive data.
 vs 	 is visual only. has semantic meaning (important text).
<i> vs 	<i> is visual only. has semantic meaning (emphasis).
div vs span	div = block-level. span = inline. Block starts new line. Inline stays on same line.
id vs class	id = unique (once per page), uses #. class = reusable, uses dot (.)
colspan vs rowspan	colspan merges COLUMNS. rowspan merges ROWS.
padding vs margin	padding = space INSIDE border. margin = space OUTSIDE border.
=== vs ==	=== checks value AND type (strict). == checks value only (loose). Use ===
var vs let vs const	var=function-scoped, let=block-scoped, const=block-scoped & cannot reassign.
null vs undefined	null = intentionally empty. undefined = variable declared but not assigned.
innerHTML vs textContent	innerHTML parses HTML. textContent treats everything as plain text.
\$_GET vs \$_POST	GET: data from URL. POST: data from form body.
echo vs print (PHP)	Both output text. print returns 1, echo is slightly faster, can output multiple.
session vs cookie	session: stored on SERVER, lost when browser closes. cookie: stored in BROWSER.
htmlspecialchars	Converts < > & ' to HTML entities. Prevents XSS attacks. Always use with user input!
target='_blank'	Opens link in new tab. target='_self' opens in same tab (default).
CSS Specificity	Inline > ID > Class > Element. !important overrides all.
position: fixed	Stays fixed on screen even when scrolling (good for navbars).
DOM	Document Object Model. Tree structure representing HTML. JS manipulates it.
PHP <?php ?>	Opening tag must be <?php (full). PHP runs on server. Browser gets only HTML output.

GOOD LUCK IN YOUR ORAL EXAM! ■

Remember: Understand concepts, don't just memorize. If asked about a tag or property, explain WHAT it does, WHY it's used, and HOW it's written.